Puzzle

Egyptian code

Puzzle No 83

This interesting fragment of an ancient Egyptian clay tablet was recently unearthed by that eminent archaeological duo — Drs Diggett and Trowell. It is thought that it represents a multiplication sum but, unless it is shown that it is possible to satisfactorily substitute digits for the symbols shown, this theory cannot be proved. There is evidence that the sum is in decimal notation and the bird symbol was used to represent the digit '3'.

Can you make the archaeological breakthrough and confirm the theory?

Solution to Puzzle No 83

As there are five digits in both the top line of the sum and the product, the first digit of the top line can only be a 1 or 2. (It cannot be a '3' as it isn't a bird symbol).

Similarly, we can see that the last digit of the top line cannot be a 0 or a 1 as this would involve a duplication of either this digit or the bird symbol in the bottom line.

The program generates all possible sets of values and tests to see that no digits are duplicated (except for the '3' in the product).

```
10 FOR A = 1 TO 2 20 FOR B = 0 TO 9 30 IF A = B
THEN GOTO 280 40 FOR C = 0 TO 9 50 IF C = A OR
C = B THEN GOTO 270 60 FOR D = 0 TO 9 70 IF D
= A OR D = B OR D = C THEN GOTO 260 80 FOR E
= 2 TO 9 90 IF E = A OR E = B OR E = C OR E = D
THEN GOTO 250 100 LET P = (A * 10000 + B *
1000 + C * 100 + D * 10 + E) * 3 110 LET P$ =
STR$ P 120 IF P$(2) <> "3" THEN GOTO 250 130
FOR M = 1 TO 4 140 FOR N = M + 1 TO 5 150 IF
P$(M) = P$(N) THEN GOTO 250 160 NEXT N 170
NEXT M 180 LET Q$ = STR$ (P/3) 190 FOR M = 1
TO 5 200 FOR N = 1 TO 5 210 IF P$(M) = Q$(N)
THEN GOTO 250 220 NEXT N 230 NEXT M 240
PRINT P$, Q$ 250 NEXT E 260 NEXT D 270 NEXT C
280 NEXT B 290 NEXT A
```

```
10 FOR A=1 TO 2
20 FOR B=0 TO 9
30 IF A=B THEN GOTO 280
40 FOR C=0 TO 9
50 IF C=A OR C=B THEN GOTO 270
60 FOR D=0 TO 9
70 IF D=A OR D=B OR D=C THEN GOTO 260
80 FOR E=2 TO 9
90 IF E=A OR E=B OR E=C OR E=D THEN GOTO 250
100 LET P = (A*10000 + B*1000 + C*100 + D*10 + E)*3
110 LET P$ = STR$(P)
120 IF P$(2) <> "3" THEN GOTO 250
130 FOR M=1 TO 4
140 FOR N=M+1 TO 5
150 IF P$(M)=P$(N) THEN GOTO 250
160 NEXT N
170 NEXT M
180 LET     Q$ = STR$(P/3)
190 FOR M=1 TO 5
200 FOR N=1 TO 5
210 IF P$(M)=Q$(N) THEN GOTO 250
220 NEXT N
230 NEXT M
240 PRINT P$,Q$
250 NEXT E
260 NEXT D
270 NEXT C
280 NEXT B
290 NEXT A
```

The Puzzle Itself explained

We must find a five-digit number, whose digits (represented by A, B, C, D, E) are all different, such that when it is multiplied by 3 the result is a five-digit number of the form:

F 3 G H I
That is, the product has its second digit fixed as 3. In addition, none of the digits in the multiplicand may appear in the product.

Why Can A Only Be 1 or 2?

1. **No Duplicates Allowed with the Fixed 3:**
   Since the product always has a "3" in its second digit, the digit 3 is "taken" there. In a cryptarithmetic puzzle, each letter (or fixed digit) must appear only once. Hence, none of the digits of the multiplicand (which make up A, B, C, D, E) may equal 3. In particular, the very first digit A (which must be nonzero) cannot be 3.
2. **Range Restrictions Based on Multiplication:**
   The multiplicand is a five-digit number. In order for its triple to also be a five-digit number (so that the product fits the F3GHI format), it must be less than 33,334 (since 33,333 × 3 = 99,999). Although from a pure size point of view A might seem that it could be 3 (as numbers starting with 3 run from 30,000 to 33,333), if A were 3 then 3 would appear in both the multiplicand (as its first digit) and in the product's fixed digit "3". That would be a violation of the all-different condition. Thus, the only choices for A that are both nonzero and not 3 are 1 and 2.

Why E can not Be 1 or 0?

Let's analyze the loop for **E**:

```
80 FOR E=2 TO 9
```

By starting at 2, the program intentionally rules out the values 0 and 1. Here's why:

1. **If E Were 0:**
   In the five-digit number ABCDE, having E = 0 means that 0 appears as one of its digits. The product is calculated as

   ```
   P = (A*10000 + B*1000 + C*100 + D*10 + E) * 3
   ```

   1. Due to modular arithmetic, the units digit of the product is determined by (E × 3) mod 10. If E is 0, then 0 × 3 is 0, so the final digit of P becomes 0. This would mean that the digit 0 is present in both the original number (as E) and the product (its last digit). Since the puzzle requires all digits to be distinct, this duplication is not allowed. Instead of letting the program check for such duplication later, the programmer preemptively avoids the issue by not allowing E to take the value 0

   2. **If E Were 1:**
      When E equals 1, the units digit of the product becomes (1 × 3) mod 10, which is 3. However, the puzzle's format for the product is F3GHI—the second digit is specifically fixed as 3. If the last digit also turns out to be 3, the product would contain the digit 3 twice. This again violates the rule that all digits must be unique within the product. To ensure that the product's fixed 3 does not get duplicated by the result of 1 × 3, the programmer starts E from 2.

Thus, by setting the loop `FOR E=2 TO 9`, the program ensures that:

- **E ≠ 0** avoids sharing the digit 0 between the multiplicand and the product.
- **E ≠ 1** prevents the product's units digit from duplicating the mandatory 3 in the product's second position.

This careful control over the range of E is a smart way of pruning out candidates that would certainly violate the uniqueness conditions later in the program.

The Algorithm and Program Flow

The program is written in Basic and uses several nested loops combined with early-exit routines (using GOTO) to enforce the conditions. Here's how it works line by line:

1. **First Loop – Setting A (Line 10 & 290):**

   ```
   10 FOR A=1 TO 2
   290 NEXT A
   ```

   - **Purpose:**
     Since A is the ten-thousands digit of the multiplicand, it must be nonzero and cannot be 3 (because 3 is reserved for the product's second digit). The loop runs only for A = 1 and 2.

2. **Second Loop – Setting B (Line 20 & 280):**

```
20 FOR B=0 TO 9
30 IF A=B THEN GOTO 280
280 NEXT B
```

- **Purpose:**
  B can be any digit from 0 to 9 except that it must differ from A. If B equals A, the program jumps to the end of the B loop (line 280).

3. **Third Loop – Setting C (Line 40 & 270):**

```
40 FOR C=0 TO 9
50 IF C=A OR C=B THEN GOTO 270
270 NEXT C
```

- **Purpose:**
  Likewise, C must be distinct from both A and B.

4. **Fourth Loop – Setting D (Line 60 & 260):**

```
60 FOR D=0 TO 9
70 IF D=A OR D=B OR D=C THEN GOTO 260
260 NEXT D
```

- **Purpose:**
  D is chosen so that it does not match any of A, B, or C.

5. **Fifth Loop – Setting E (Line 80 & 250):**

```
80 FOR E=2 TO 9
90 IF E=A OR E=B OR E=C OR E=D THEN GOTO 250
250 NEXT E
```

- **Purpose:**
  E is chosen from 2 through 9 (note that 0 and 1 are not allowed here—1 is possibly already used by A in one branch, and 3 is never allowed because it's the product's fixed digit). This further limits the candidate numbers so that all five digits are different.

6. **Compute the Product (Line 100):**

```
100 LET P = (A*10000 + B*1000 + C*100 + D*10 + E)*3
```

- **Purpose:**
  This forms the five-digit number ABCDE, multiplies it by 3, and stores the result in P.

7. **Convert the Product to a String (Line 110):**

```
110 LET P$ = STR$(P)
```

- **Purpose:**
  The string conversion is used so that individual digit characters can be tested.

8. **Check the Fixed 3 Condition (Line 120):**

```
120 IF P$(2) <> "3" THEN GOTO 250
```

- **Purpose:**
  The program then checks that the product's second character is "3". (In this Basic dialect, the conversion produces a string with the first character being the most significant digit.) This enforces the pattern F3GHI.

9. **Ensure the Product's Digits Are All Different (Lines 130–170):**

```
130 FOR M=1 TO 4
140 FOR N=M+1 TO 5
```

```
150 IF P$(M)=P$(N) THEN GOTO 250
160 NEXT N 170 NEXT M
```

- **Purpose:**
  These nested loops compare every pair of digits in the product to ensure there are no duplicates
  .

10. **Ensure Multiplicand and Product Share No Digits (Lines 180–230):**

```
180 LET Q$ = STR$(P/3)
190 FOR M=1 TO 5
200 FOR N=1 TO 5
210 IF P$(M)=Q$(N) THEN GOTO 250
220 NEXT N
230 NEXT M
```

- **Purpose:**
  Here, Q$ is simply the string form of the original five-digit number (since P/3 = ABCDE). The nested loops then check that none of the digits in the multiplicand appear in the product, adhering to the rule that all digits must be unique across the entire problem.

11. **Print the Valid Solution (Line 240):**

```
240 PRINT P$,Q$
```

- **Purpose:**
  If the candidate passes all tests, its product (P$) and the original number (Q$) are printed.

In Summary

- **Why A is Restricted to 1 or 2:**
  Because the puzzle's requirements force the entire multiplication to use all different digits and specifically use a fixed digit "3" in the product's second position. Since A is the leading digit of the multiplicand and must not be 0 or duplicate the fixed digit 3, the only possibilities left (in the range that ensures the product stays a five-digit number) are 1 and 2.
- **Overall Program Flow:**
  The program uses five nested loops to assign digits to A, B, C, D, and E—ensuring all are distinct. It then calculates the product by multiplying by 3. It converts the product to a string so it can check that the second digit is "3" (to meet the F3GHI pattern) and that all digits in the product are unique. It finally confirms that no digit appears in both numbers before printing the solution. Every time a condition fails, the program uses a GOTO to jump out of the inner loops and try the next candidate.

Additional Thoughts

Such puzzles—often called *cryptarithmetic* or *alphametic puzzles*—challenge one to assign unique digits to letters so that a mathematical operation holds true. A programmer can greatly narrow the search space (here by precluding A from being 3) by noticing in advance that the fixed digit "3" in the product disallows its reuse in the multiplicand. This clever insight not only speeds up the search but also reduces unnecessary iterations.
If you're interested, you might explore variants where different digits are fixed in the result, or even puzzles where the multiplier is unknown. Each slight change alters the way you "prune" the possibilities.
I hope this detailed walkthrough clarifies both the reasoning behind choosing A = 1 or 2 and the complete flow of the Basic program.

According to Microsoft Copilot, an AI companion developed by Microsoft